

PRUEBA TÉCNICA

Aprendiz / Practicante · Backend Developer

Proyecto: **Sistema de Gestión de Tareas** — Node.js + Express + TypeScript

El objetivo de esta prueba es desarrollar una API RESTful robusta y bien organizada que demuestre habilidad para escribir código limpio, estructurado y seguro, aplicando prácticas de desarrollo backend con TypeScript. La API permitirá a los usuarios registrarse, iniciar sesión y administrar sus propias tareas personales.

Tecnologías

- Node.js
 - Express.js
 - TypeScript
 - PostgreSQL (preferido) o MongoDB
-

1. Estructura del Proyecto y Configuración

Se evaluará no solo la funcionalidad final, sino también la estructura y calidad del código.

Organización en Capas

Estructurar el proyecto en carpetas que separen claramente las responsabilidades:

- **src/api** — Definición de rutas (routes) y middlewares.
- **src/controllers** — Lógica principal que atiende las peticiones.
- **src/services** — Lógicas de negocio más complejas o reutilizables.
- **src/persistence** — Comunicación con la base de datos.

En caso de ser necesario, añadir otras carpetas (utils, config, etc.).

Gestión de Configuración

- Utilizar un archivo **.env** para gestionar variables de entorno (puerto, datos de conexión, etc.). Se debe incluir un archivo **.env.example** en el repositorio.
 - Para la gestión de variables dentro de la aplicación, se sugiere un patrón **Singleton** que las cargue una sola vez y las ponga disponibles donde se necesiten.
-

2. Autenticación y Autorización Sencilla

Registro

Crear un endpoint **POST /auth/register** que permita a un nuevo usuario registrarse con nombre, email y contraseña. La contraseña nunca debe guardarse en texto plano; utilizar una librería como **bcrypt** para generar un hash seguro.

Inicio de Sesión

Implementar un endpoint **POST /auth/login** que verifique las credenciales del usuario. Si son correctas, debe generar y devolver un **JSON Web Token**.

Protección de Rutas

Desarrollar un middleware que intercepte las peticiones a rutas protegidas. Debe leer el token desde la cabecera **Authorization: Bearer <token>**, verificar su validez y, si es correcto, permitir el acceso.

3. Gestión de Tareas (CRUD)

Cada tarea debe tener: **id**, **titulo**, **descripcion**, **fecha_vencimiento** y **estado** ('pendiente', 'en curso', 'completada'). Muy importante: cada tarea debe estar asociada al usuario que la creó.

- **POST /tasks** — Permite a un usuario autenticado crear una nueva tarea.
 - **GET /tasks** — Devuelve la lista de tareas que pertenecen únicamente al usuario autenticado.
 - **GET /tasks/:id** — Permite a un usuario consultar una tarea específica. Solo el dueño puede acceder.
 - **PUT /tasks/:id** — Permite a un usuario modificar una de sus propias tareas.
 - **DELETE /tasks/:id** — Permite a un usuario eliminar una de sus propias tareas.
-

4. Validación y Manejo de Errores

Validación de Entradas

Asegurarse de que los datos que llegan a la API son correctos (ej. que el email tenga formato válido). Se recomienda usar esquemas con una librería como **AJV con JSONSchema** o similar.

Manejo de Errores Centralizado

Crear clases de error personalizadas (ej. **NotFoundError**, **AuthenticationError**) para representar diferentes situaciones, identificarlas y enviar una respuesta JSON clara y consistente al usuario.

5. Documentación y Buenas Prácticas

- **Swagger** — Documentar todos los endpoints directamente en el proyecto.
 - **JSDoc** — Documentar las funciones y clases más importantes. El código debe ser legible y seguir las mejores prácticas de TypeScript.
 - **Conventional Commits** — Realizar commits atómicos y descriptivos (ej. feat: add user login endpoint, fix: correct task validation schema).
-

Entregables

Código Fuente

Repositorio en GitHub con instrucciones claras en el **README.md**, que debe incluir:

- Descripción del proyecto y su arquitectura.
- Instrucciones detalladas de instalación local (npm install).
- Cómo configurar el archivo .env.
- Comandos para ejecutar el proyecto (npm run dev).

Bitácora de Desarrollo — DEVELOPMENT_LOG.md

Este archivo documenta el proceso de desarrollo. Tiene una sección especialmente importante relacionada con el uso de herramientas de inteligencia artificial:

Uso de Asistentes de IA (Obligatorio si se usaron)

El objetivo de esta sección es demostrar criterio y pensamiento crítico frente al código generado. Se evaluará la capacidad del candidato para juzgar, cuestionar y apropiarse del resultado, no solo para obtenerlo.

Para cada uso significativo de IA, se debe documentar:

- **El prompt utilizado** — Exactamente cómo se formuló la solicitud.
- **Qué se aceptó y por qué** — Explicar qué partes del código generado se incorporaron y qué las hace correctas o adecuadas para el proyecto.
- **Qué se rechazó o modificó** — Describir qué fragmentos no se aceptaron directamente, indicando el motivo (seguridad, tipado incorrecto, lógica equivocada, etc.).
- **Verificación realizada** — Explicar cómo se comprobó que el código era correcto, seguro y coherente con el resto de la aplicación.
- **Decisiones sin asistencia de IA** — Señalar al menos dos decisiones arquitectónicas o de implementación que se tomaron de forma independiente, explicando el razonamiento detrás de ellas.

Nota: Un candidato que usa IA con criterio —entendiendo lo que produce— es más valioso que uno que escribe todo sin asistencia sin comprenderlo. Esta sección busca evidenciar exactamente eso.

Retos y Soluciones

Describir brevemente los bloqueos o errores significativos encontrados durante el desarrollo y cómo se superaron.

Deseables (No obligatorios)

- Pruebas unitarias y/o de integración con **Vitest** o **Jest**.
- Despliegue de la aplicación en un servicio online (Heroku, Vercel, AWS, etc.).

Criterios de Evaluación

#	Criterio	Descripción
1	Cumplimiento de Requisitos Funcionales y Arquitectónicos	Si la aplicación cumple con todo lo solicitado y respeta la estructura en capas definida.
2	Calidad del Código y Adherencia a Patrones de Diseño	Uso correcto de TypeScript, código limpio, modular y la implementación de los patrones solicitados (configuración centralizada, manejo de errores, etc.).
3	Claridad y Calidad de la Documentación	La calidad del README.md, la documentación de Swagger y los comentarios JSDoc.
4	Uso Adecuado de Git	La calidad y frecuencia de los commits.
5	Análisis de la Bitácora de Desarrollo	La capacidad para documentar el proceso, resolver problemas y utilizar herramientas de IA de manera efectiva.
6	Uso Crítico de Herramientas de IA	Se evalúa si el candidato demuestra comprensión real del código producido, independientemente de su origen. Esto incluye: capacidad para identificar errores en código generado por IA, justificación de las decisiones tomadas con o sin asistencia, y evidencia de que el candidato entiende y puede defender cada parte de su solución.

Plazo de Entrega

4 días calendario desde la recepción de este documento.

Instrucciones Finales: Enviar el enlace al repositorio de GitHub y el video explicativo.
